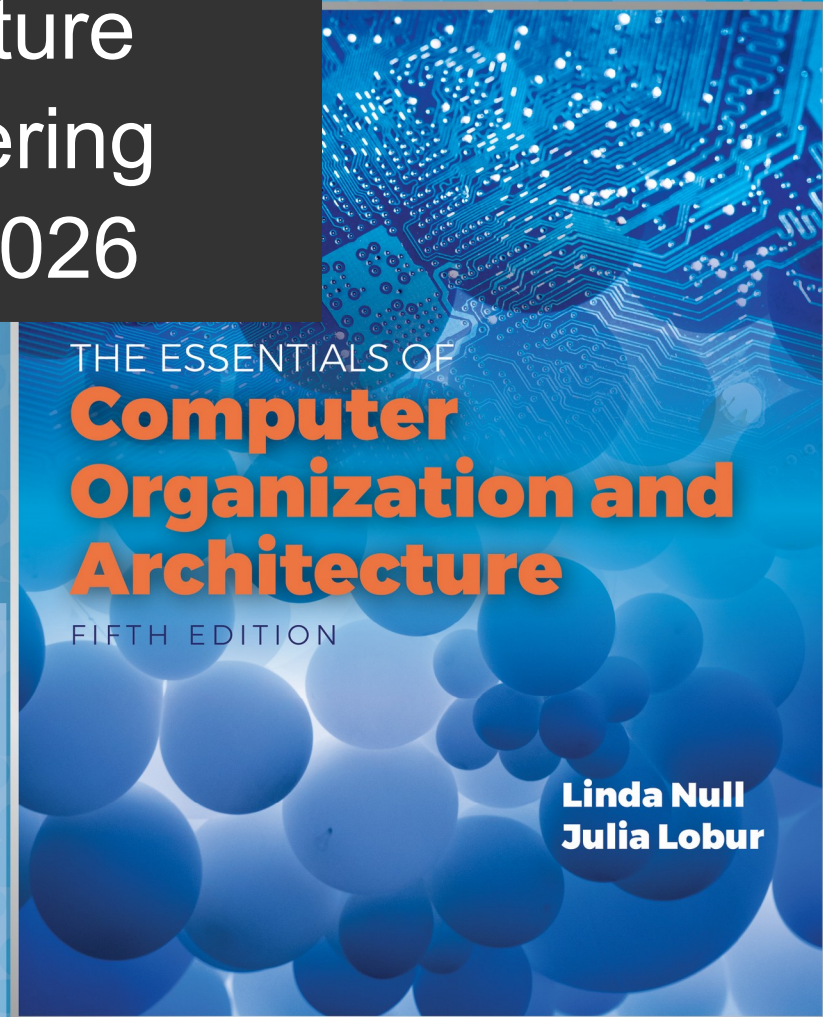


Computer Architecture Informatics Engineering ESTiG/IPB, 2025/2026

José Rufino

(mainly based on Chapter 11 of the book “The Essentials of Computer Organization and Architecture” of Linda Null & Julia Lobur)

Unit 7: Performance Measurement and Optimization



Objectives



- Understand the ways in which computer performance is measured.
- Be able to describe common benchmarks and their limitations.
- Become familiar with factors that contribute to improvements in CPU performance.
- Know how to evaluate the impact of each component on the global system performance.

7.1 Introduction



- The ideas presented will help to understand various measurements of computer performance.
- It will be useful to use these ideas when buying a large system, or trying to improve the performance of an existing system.
- A number of factors will be discussed that affect system performance, including some tips that may be used to improve the performance of programs.

7.2 Mathematical Tools



- System performance measurement and optimization depend upon one's point of view and main focus.
- A **computer user** is mostly concerned with ***response time*** (how long does the system take to react to the user) and ***turnaround time*** (how long does it take to carry out a task).
 - these two times should be minimized
- **System administrators** mostly concern about ***throughput*** (how many tasks can the system handle – concurrently or in parallel – before response time is adversely affected?).
 - this throughput should be maximized
- Performance optimization seeks to reconcile these two (apparently incompatible) requirements.

7.2 Mathematical Tools

- Speedup



- to compare the performance of two systems with base on a certain task, measure the time taken to execute it
- if **T(A)** and **T(B)** is the time spent by system A and B to do the same task, their relative performance is given by:

$$S(A,B) = T(B)/T(A)$$

= *speedup* of A relative to B

$$S(B,A) = T(A)/T(B)$$

= *speedup* of B relative to A

- $[S(X,Y) - 1] \times 100 = m\%$

- if $S(X,Y) > 1$, then $m > 0$ and X is **m%** faster than Y
- if $S(X,Y) < 1$, then $m < 0$ and X is **|m%|** slower than Y
(or, equivalently, X has $S(X,Y)$ of Y's performance)

7.2 Mathematical Tools

- Speedup



- example:

checkpoint: exercise FM1

- system A executes a program in 3 minutes, and system B executes the same program in 4 minutes
- the speedup of A relative to B is **1.33** (>1), meaning A is **33% faster** than B when running that same program

$$S(A,B) = \frac{T(B)}{T(A)} = \frac{4}{3} = \mathbf{1.33}$$

$$(\mathbf{1.33} - 1) \times 100 = \mathbf{33\%}$$

- the speedup of B relative to A is **0.75** (<1), meaning B is **25% slower than** A (or, equivalently, **B exhibits 75% of A's performance** when running that same program)

$$S(B,A) = \frac{T(A)}{T(B)} = \frac{3}{4} = \mathbf{0.75}$$

$$(\mathbf{0.75} - 1) \times 100 = \mathbf{-25\%}$$

7.2 Mathematical Tools

- Arithmetic Mean



- To evaluate system performance, typically is executed a **set of representative tasks**, which define a **workload**
- Each task will have a **specific performance measure**
- These measures may be combined in a single **average**
- The most well-known average is the ***arithmetic mean***

$$\frac{\sum_{i=1}^n x_i}{n}$$

7.2 Mathematical Tools - Arithmetic Mean

- But the arithmetic mean can be misleading (specially if the data are skewed or scattered)
 - Consider the execution times given in the table below. The performance differences are hidden by the simple average.
 - **All three systems seem to perform the same, once the three arithmetic means have exactly the same value.**

Program	System A Execution Time	System B Execution Time	System C Execution Time
v	50	100	500
w	200	400	600
x	250	500	500
y	400	800	800
z	5000	4100	3500
Average	1180	1180	1180

7.2 Mathematical Tools

- Weighted Arithmetic Mean

- If execution frequencies (expected workloads) are known, a ***weighted arithmetic mean*** (or simply ***weighted average***) is more adequate (and revealing) than the normal average
 - For the frequencies bellow, the weighted arithmetic mean for System A is $50 \times 0.5 + 200 \times 0.3 + 250 \times 0.1 + 400 \times 0.05 + 5000 \times 0.05 = 380$ and for System C is $500 \times 0.5 + 600 \times 0.3 + \dots + 3500 \times 0.05 = 695$

Program	Execution Frequency	System A Execution Time	System C Execution Time
v	50%	50	500
w	30%	200	600
x	10%	250	500
y	5%	400	800
z	5%	5000	3500
Weighted Average		380 seconds	695 seconds

- It can now be said that system A is **83%** faster than System C for this particular workload ($695 / 380 = 1,83$)

7.2 Mathematical Tools

- Weighted Arithmetic Mean

- However, workloads can change over time.
 - A system optimized for one workload may perform poorly when the workload changes, and the ranking of the compared systems may even switch, as illustrated below.

Program	Execution Frequency	System A Execution Time	System C Execution Time
v	25%	50	500
w	5%	200	600
x	10%	250	500
y	5%	400	800
z	55%	5000	3500
Weighted Average		2817.5 seconds	2170 seconds

- It can now be said that system C is **30%** faster than System A for this particular workload ($2817,5 / 2170 \approx 1,30$)

checkpoint:
exercise FM2

7.2 Mathematical Tools

- Geometric Mean



- Arithmetic average should not be used if data is scattered/skewed.
- Weighted average is adequate for static and representative workloads.
- The **geometric mean** gives us a consistent number with which to perform comparisons regardless of the distribution of the data.
 - vs the arithmetic mean: the geometric mean gives more weight to the smaller samples (x_n), is less affected by sampling fluctuations
 - its value is “**the n th root of the product of n measurements**”.

$$G = \left[x_1 \cdot x_2 \cdot x_3 \cdot \dots \cdot x_n \right]^{\frac{1}{n}}$$

7.2 Mathematical Tools

- Geometric Mean

- The geometric mean is typically computed based on **normalized** ratios between a **reference machine** and a system under test.
 - below, execution times are normalized based on **system B**

Program	System A Execution Time	Execution Time Normalized to B	System B Execution Time	Execution Time Normalized to B	System C Execution Time	Execution Time Normalized to B
v	50	2	100	1	500	0.2
w	200	2	400	1	600	0.6667
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0.82	4100	1	3500	1.1714
Geometric Mean		1.6733		1		0.6898

$$\frac{\text{System B Execution Time}}{\text{System A Execution Time}}$$

$$\frac{\text{System B Execution Time}}{\text{System C Execution Time}}$$

7.2 Mathematical Tools

- Geometric Mean

- As expected, when another system is used for a reference machine, we get a different set of numbers
 - below, execution times are normalized based on **system C**

Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	50	10	100	5	500	1
w	200	3	400	1.5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0.7	4100	0.8537	3500	1
Geometric Mean		2.4258		1.4497		1

$$\frac{\text{System C Execution Time}}{\text{System A Execution Time}}$$

$$\frac{\text{System C Execution Time}}{\text{System B Execution Time}}$$

7.2 Mathematical Tools

- Geometric Mean



- But, no matter which system is used as a reference, **the ratio of the geometric means is consistent.**
 - This can be clearly seen bellow, when using either System B or System C as reference machines

System A Execution Time	Execution Time Normalized to B	System B Execution Time	Execution Time Normalized to B	System C Execution Time	Execution Time Normalized to B
Geometric Mean	1.6733	1		0.6898	

$$\begin{aligned} 1,6733 / 1 = \\ 2,4258 / 1,4497 \end{aligned}$$

System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
Geometric Mean	2.4258	1.4497		1	

$$\begin{aligned} 1 / 0,6898 = \\ 1,4497 / 1 \end{aligned}$$

7.2 Mathematical Tools

- Geometric Mean



- Based on the geometric means, or their ratios, it can be defined a ranking of the systems by their performance
 - for the previous example, it can be said that A is the fastest system, followed by B, and then C - a relative comparison
- But geometric means, or their ratios, are not speedups
 - for the previous example, although the ratio of the geometric means of System C to System A is ≈ 2.43 , this does not imply that System A is ≈ 2.43 times as fast as System C
- So, unlike arithmetic means, the geometric mean does not convey a value of absolute system performance. It is only a tool for comparison, to define a ranking of the systems.

7.2 Mathematical Tools

- Geometric Mean



- A inherent problem of using the geometric mean to evaluate and compare machine performance is that all execution times contribute equally to the result.
- So, shortening the execution time of a small program by 10% has the same effect as shortening the execution time of a large program by 10% (see →)
 - Shorter programs are generally easier to optimize, but in the real world we want to shorten the execution time of longer programs.

7.2 Mathematical Tools

- Geometric Mean



Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	45	10	90	5	450	1
w	200	3	400	1,5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0,7	4100	0,8537	3500	1

Geometric
Mean

2,4258

1,4497

1,0000

Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	50	10	100	5	500	1
w	200	3	400	1,5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	4500	0,7	3690	0,8537	3150	1

Geometric
Mean

2,4258

1,4497

1,0000

- optimizing v spares $5 + 10 + 50 = 65s$
- optimizing z spares $500 + 410 + 350 = 1260s$ (almost 20x more)
- however, the geometric mean remains the same

7.2 Mathematical Tools

- Weighted Geometric Mean

- the **geometric mean** also has a **weighted** variant
 - each rate x_i is raised to a specific weight w_i
 - assuming that $w_1 + w_2 + w_3 + \dots + w_n = 1$ then

$$G = (x_1^{w_1} * x_2^{w_2} * x_3^{w_3} * \dots * x_n^{w_n})$$

- this formulation is adequate when the importance of each term x_i (coming from a different benchmark) is different
- for $w_i = 1/n$, we get the non-weighted geometric mean
- see also <https://dl.acm.org/doi/pdf/10.1145/5666.5673>

checkpoint: exercise FM3

7.2 Mathematical Tools

- Harmonic Mean



- Neither the geometric mean nor the arithmetic mean is appropriate when the data (x_n) is expressed as a **rate**
 - rate (or throughput) = number of operations per second
- The **harmonic mean** allows us to form a mathematical expectation of throughput, and to compare the relative throughput of systems and system components.
- To find the harmonic mean, we add the reciprocals of the rates and divide them into the number of rates:

$$H = n \div (1/x_1 + 1/x_2 + 1/x_3 + \dots + 1/x_n)$$

7.2 Mathematical Tools - Harmonic Mean



- example:
 - a program A with 3000 instructions executed at a rate of 3000 instr./us during the first **1000** instructions, at the rate of 4000 instr./us during the second **1000** instructions, and at the rate of 6000 instr./us during the last **1000** instructions
 - arithmetic mean of the rates: **4333.3(3)** instr./us (WRONG !!)
 - geometric mean of the rates: **4160.17** instr./us (WRONG !!)
 - execution time of the 1st 1000 instructions: $1000/3000 = 1/3$ us
 - execution time of the 2nd 1000 instructions: $1000/4000 = 1/4$ us
 - execution time of the 3rd 1000 instructions: $1000/6000 = 1/6$ us
 - total execution time: $\frac{3}{4}$ us; **average execution rate** = 3000 instr. / $\frac{3}{4}$ us = **4000** instr./us ; **harmonic mean** = $3 / (1/3000 + 1/4000 + 1/6000) = \mathbf{4000}$ instr./us (RIGHT !!)

7.2 Mathematical Tools

- Harmonic Mean



- the **harmonic mean** also has a **weighted** variant
 - each rate reciprocal ($1/x_i$) is affected by a weight w_i
 - assuming that $w_1 + w_2 + w_3 + \dots + w_n = 1$ then
$$H = 1 \div (w_1/x_1 + w_2/x_2 + w_3/x_3 + \dots + w_n/x_n)$$
 - this formulation is adequate when the number of samples or events is different for each rate x_i
 - $w_i = (\text{n}^\circ \text{ of events at rate } x_i) / (\text{total n}^\circ \text{ of events})$
 - the following slide shows a situation like this
 - see also https://en.wikipedia.org/wiki/Harmonic_mean

7.2 Mathematical Tools

- Harmonic Mean

checkpoint:
exercises
FM4A/B/C

- example with weighted harmonic mean:
 - a program A with 3000 instructions executed at a rate of 3000 instr./us during the first **1500** instructions, at the rate of 4000 instr./us during the next **1000** instructions, and at the rate of 6000 instr./us during the last **500** instructions
 - execution time of the 1st 1500 instructions: $1500/3000 = 0.5$ us
 - exec. time of the next 1000 instructions: $1000/4000 = 0.25$ us
 - exec. time of the last 500 instructions: $500/6000 = 0.083(3)$ us
 - total execution time: $0.5 + 0.25 + 0.083(3) = 0.83(3)$ us; **average execution rate** = $3000 \text{ instr.} / 0.83(3) \text{ us} = \mathbf{3600}$ instr./us ;
weighted harmonic mean = $1 / [(1500/3000)/3000 + (1000/3000)/4000 + (500/3000)/6000] = \mathbf{3600}$ instr./us; the weights w_i are the proportion of instructions executed at different rates

7.2 Mathematical Tools - Harmonic Mean



- **Advantages** of the harmonic over the geometric mean
 - it is a suitable predictor of machine behavior.
 - it is useful for more than simply comparing performance.
 - the smallest rates have the greatest influence on the result, so improving the slowest programs results in better performance (this is what makes sense in the real world)
- As with the geometric mean, the harmonic mean can be used with relative performance ratios, but is more sensitive to the choice of the reference machine.
 - That is, the ratios of the harmonic means are not as consistent as those of the geometric means

7.2 Mathematical Tools

- Sum Up



- This chart summarizes when the use of each of the previous performance means is appropriate.

Mean	Uniformly Distributed Data	Skewed Data	Data Expressed as a Ratio	Indicator of System Performance Under a Known Workload	Data Expressed as a Rate
Arithmetic	X			X	
Weighted Arithmetic				X	
Geometric		X	X		
Harmonic				X	X

7.2 Mathematical Tools

- Statistics and Semantics



- The objective assessment of computer performance is most critical when deciding which one to buy.
 - For enterprise-level systems, this process is complicated, and the consequences of a bad decision may be grave.
- Unfortunately, computer sales are as much dependent on good marketing as on good performance.
- The wary buyer will understand how objective performance data can be slanted to the advantage of anyone giving a sales pitch (a biased presentation).

7.2 Mathematical Tools

- Statistics and Semantics



- The most common deceptive practices include:
 - Selective statistics: citing only favorable results
 - Citing only *peak* performance (ignoring the average case)
 - Vagueness in the use of words like “almost,” “nearly,” “more,” and “less,” in comparing performance data
 - Use of inappropriate statistics (“comparing apples to oranges”)
 - Implying that you should buy a particular system because “everyone” is buying similar systems
 - Changing system behavior when benchmarking is detected

7.3 Benchmarking



- Performance benchmarking is “the science of making objective assessments concerning the performance of one system over another”. These evaluations are also important to measure the effect of SW/HW updates.
- *Price-performance ratios* can be derived from standard benchmarks.
- The troublesome issue is that there is no definitive benchmark that can tell you which system will run *your* applications the fastest (using the least *wall clock* time) for the least amount of money.

7.3 Benchmarking - CPU Speed



- **CPU speed** often assumed to equate to system performance
- Measures of CPU speed include **cycle time** (MHz, and GHz) and **millions of instructions per second (MIPS)**.
- However, saying that System A is faster than System B because System A runs at 1.4GHz and System B runs at 900MHz is valid only when the ISAs (Instruction Set Architectures) of Systems A and B are identical.
 - although, with different ISAs it is possible that both of these systems could obtain identical results within the same wall clock time

7.3 Benchmarking - CPU Speed



- the metric **MIPS** involves the execution rate of a mix of integer, floating point and logical operations
- however, different ISAs require a different number of clock cycles for those instruction and that is not taken into account by the MIPS metric
- **example:** a CISC system executes a division using 20 instructions; a RISC system needs 60 instructions; division takes one second in both; the MIPS metric on the RISC system will be the triple of the same metric in the CISC system, which is very misleading !

7.3 Benchmarking

- CPU Speed



- Something similar occurs with the metric **MFLOPS** (*millions of floating point operations per second*), originally conceived for supercomputers.
- In fact, there's not even a consensus about a FLOP:
 - if a product builds on several sums, is the number of these sums accounted for ? if it is, then more efficient implementations (less sums) will be underrated !
- Despite these limitations, the cycle time, and the MIPS and MFLOPS metrics are very useful to compare systems of the same ISA or brand ...

7.3 Benchmarking

- Total Execution Time



- how to measure in Linux
 - **time .\x.exe**
real 0m0.081s // wall-clock time (total time)
user 0m0.072s // CPU time (x.exe is executing)
sys 0m0.009s // non-CPU time (x.exe is blocked)
- how to measure in Windows
 - **powershell "Measure-Command{.\x.exe | Out-Default}")**
TotalMilliseconds : 80.3549
- in both situations: time resolution limited to milliseconds,
reliable only to measure slow / not very fast programs

7.3 Benchmarking

- Partial Execution Time



```
#include <stdio.h>
#include <sys/time.h>
int main () {
    // Start measuring time
    struct timeval begin, end;
    gettimeofday(&begin, 0);
    // Task
    int iterations = 1000*1000*1000;
    double sum = 0, add = 1;
    for (int i=0; i<iterations; i++) {
        sum += add; add /= 2.0;
    }
    // Stop measuring time
    gettimeofday(&end, 0);
    // Calculate the elapsed time
    long seconds = end.tv_sec - begin.tv_sec;
    long microseconds = end.tv_usec - begin.tv_usec;
    double elapsed = seconds + microseconds*1e-6;

    printf("Result: %.3f\n", sum);
    printf("Time: %.3f seconds.\n", elapsed);
    return 0;
}
```

- to measure the **duration of parts of the code**, or very fast programs, we can use functions native of the programming language or primitives of the operating system
- more examples in (*)

checkpoint:
activity TE1

7.3 Benchmarking

- Partial Execution Time



- another way is to take advantage of the RDTSC (Read Time-Stamp-Counter) machine instruction in x86 CPUs
- the TSC register counts the n° of CPU cycles since boot
- the number of cycles is easily convertible to time (as long as the CPU frequency remains constant)
 - **#seconds** = #cycles / frequency (Hz)
- in CPUs with cores with different frequencies (big vs LITTLE), and dynamic frequencies (speedstep), using this approach requires some further correction steps
- the next slide shows a simple example of RDTSC use

7.3 Benchmarking

- Partial Execution Time

```
#include <stdio.h>
#include <stdlib.h>
#include <asm/types.h>

int main(){
    static __u32 hi1, hi2, lo1, lo2; __u64 interval;

    // Start measuring time
    asm("rdtsc; movl %%edx, %0; \
    movl %%eax, %1" : "=r" (hi1), "=r" (lo1) : : "%edx", "%eax");

    // Task
    { long a; a=random(); }
    //{ long a; scanf("%ld", &a); }

    // Stop measuring time
    asm("rdtsc; movl %%edx, %0; \
    movl %%eax, %1" : "=r" (hi2), "=r" (lo2) : : "%edx", "%eax");

    interval = (((__u64)hi2 << 32) + lo2) - (((__u64)hi1 << 32) + lo1);

    printf("Cycles: %llu\n", interval);
    printf("Time(s): %.9f\n", (double)interval/ (4*1e9));
}
```

Linux only

4GHz =
 4×10^9 Hz

7.3 Benchmarking

- Number of Samples, Environment

- execution time should be measured ... several times !
- how many samples ? how to handle them ?
 - **k-best**: lowest of k values (k samples)
 - **k-best mean**: average of the lowest k-values
 - **mean**: arithmetic average of all values
 - what if the Standard Deviation is “high” ?
 - **median**: middle value when values are sorted
 - **mode**: more frequent value
 - but repetition is extremely unlikely for very small values
 - ignore the 1s value ? (and why ?)
- benchmark conditions: reboot between samples ?
wait for system to become idle ? is this fair / realistic ?

7.3 Benchmarking

- Synthetic Benchmarks



- In an effort to describe performance independent of clock speed and ISAs, a number of *synthetic benchmarks* have been attempted over the years.
- Synthetic benchmarks are programs that serve no purpose except to produce performance numbers.
- The earliest synthetic benchmarks, *Whetstone*, *Dhrystone*, and *Linpack* (to name only a few) were relatively small programs that were easy to optimize.
 - This fact limited their usefulness from the outset.
- These programs are much too small to be useful in evaluating the performance of today's systems.

7.3 Benchmarking

- Synthetic Benchmarks



- Standard Performance Evaluation Corporation (SPEC, 1988): formed to address the need for objective benchmarks.
- SPEC produces benchmark suites for various classes of computers and computer applications.
- Most SPEC benchmarks aren't free (must be bought).
- A SPEC benchmark is a set of small programs/**kernels**
 - Kernels carry out the core steps in solving a particular problem.
 - Activities that do not contribute to solving the problem, such as I/O in CPU-bound problems, are removed.

7.3 Benchmarking

- Synthetic Benchmarks



- the most known SPEC benchmark is the **SPEC CPU**
- the last version (2026) has 4 suites with 52 benchmarks
[<https://www.spec.org/cpu2026/docs/overview.html#benchmarks>]
- these *benchmark* are coded in C, C++ and Fortran, covering a great variety of application fields
- evaluate **CPU**, **Memory** and **Compilers *performance***
- generate metrics like **SPECspeed** and **SPECrate**
[<https://www.spec.org/cpu2026/docs/overview.html#Q15>]
 - SPECspeed: *time* to execute **1 instance**
 - SPECrate: runs **many instances in parallel**; measures *throughput* (number of instances concluded per u.t.)

7.3 Benchmarking

- Synthetic Benchmarks



- The execution time of each kernel is divided by the execution time of the same kernel in a reference machine (Lenovo ThinkSystem HR330A with an eMAG CPU of the ARMv8 ISA, 32 cores, and 3.3 GHz turbo frequency).
[<https://www.spec.org/cpu2026/docs/overview.html#Q18>]
- Final result: **geometric mean** of all the kernel ratios
- Manufacturers may report two sets of numbers: the **peak** and **base** numbers are the results with and without compiler optimization flags, respectively.
[<https://www.spec.org/cpu2026/docs/overview.html#Q16>]

7.3 Benchmarking

- Synthetic Benchmarks



- SPEC CPU evaluates CPU+RAM performance.
- When measuring the performance of the entire system under high transaction loads, the *Transaction Performance Council* (TPC) benchmarks are more suitable.
 - the I/O activity (storage, network) is not taken into account
- The current version of this benchmark is TPC-C v5.
[<https://www.tpc.org/tpcc/default5.asp>]
- TPC-C models the transactions typical of a ware-house and distribution business using terminal emulators.

7.3 Benchmarking

- Synthetic Benchmarks



- Transactions are simulated with random generated data in order to minimize the impact of caches.
- 90% of transactions should complete in less than 5s.
- The TPC-C metric is the n° of new warehouse order transactions per minute (*tpmC*), while a mix of other transactions is concurrently running on the system.
- The *tpmC* result is divided by the total cost of the configuration tested to give a price-performance ratio.
- The price of the system includes all hardware, software, and maintenance fees that the customer expects to pay.

7.3 Benchmarking

- Synthetic Benchmarks



- The *TPC Council* has also devised benchmarks for decision support (data mining) and WWW e-commerce systems.
[<https://www.tpc.org/information/benchmarks5.asp>]
- Results of the tests are audited by an independent auditing firm that has been certified by the TPC.
- For all of the TPC benchmarks, the systems tested must be available for general sale at the time of the test and at the prices cited in a full disclosure report.
- TPC benchmarks are a kind of simulation tool that can be used to optimize system performance in extreme conditions

checkpoint: activity BS1

7.4 CPU Performance Optimization

- CPU optimization includes many of the topics that have been covered in preceding chapters.
 - integrated *floating-point units* (improve on FP emulation)
 - *pipelining* (including delayed branching and branch prediction, to keep the pipeline full)
 - *super-pipelining* (>1 instruction finished per pipeline clock cycle), *superscalarity* (>1 execution unit, e.g. ALU+FPU, allow executing 2 instructions per CPU clock cycle)
 - CPU *caches*, *multi-core* CPUs (see unit 8), ...
- yet to explore: **code optimization techniques**; involves manual (user) or automatic (compiler) optimizations
 - *loop unrolling, loop fusion, loop fission, data (in)dependence, loop interchange, function call avoidance, ...*

7.4 CPU Performance Optimization

- Code Optimization Techniques

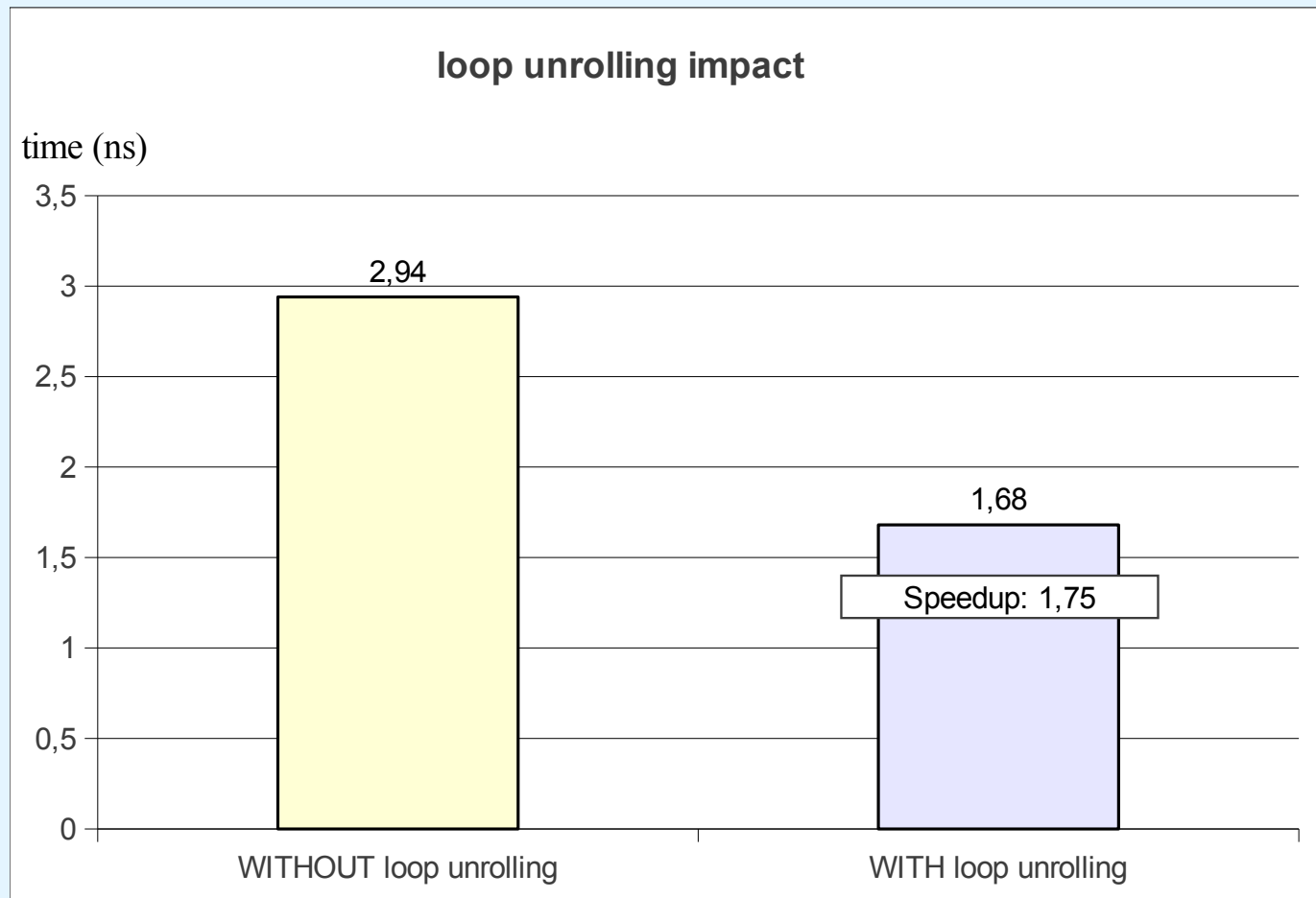
- ***Loop unrolling***: expands a loop so that each new iteration contains several of the original operations, thus performing more computations per iteration.

```
// WITHOUT loop unrolling
for (i=0; i<3000; i++)
    A[i] = A[i] + B[i] * c;
```

```
// WITH loop unrolling
for (i=0; i<3000; i+=3){
    A[i] = A[i] + B[i] * c;
    A[i+1] = A[i+1] + B[i+1] * c;
    A[i+2] = A[i+2] + B[i+2] * c;
}
```

7.4 CPU Performance Optimization

- Code Optimization Techniques



gcc options -O0 -lm, Core i7-8865U 4.8GHz 14 nm (2018)

7.4 CPU Performance Optimization

- Code Optimization Techniques

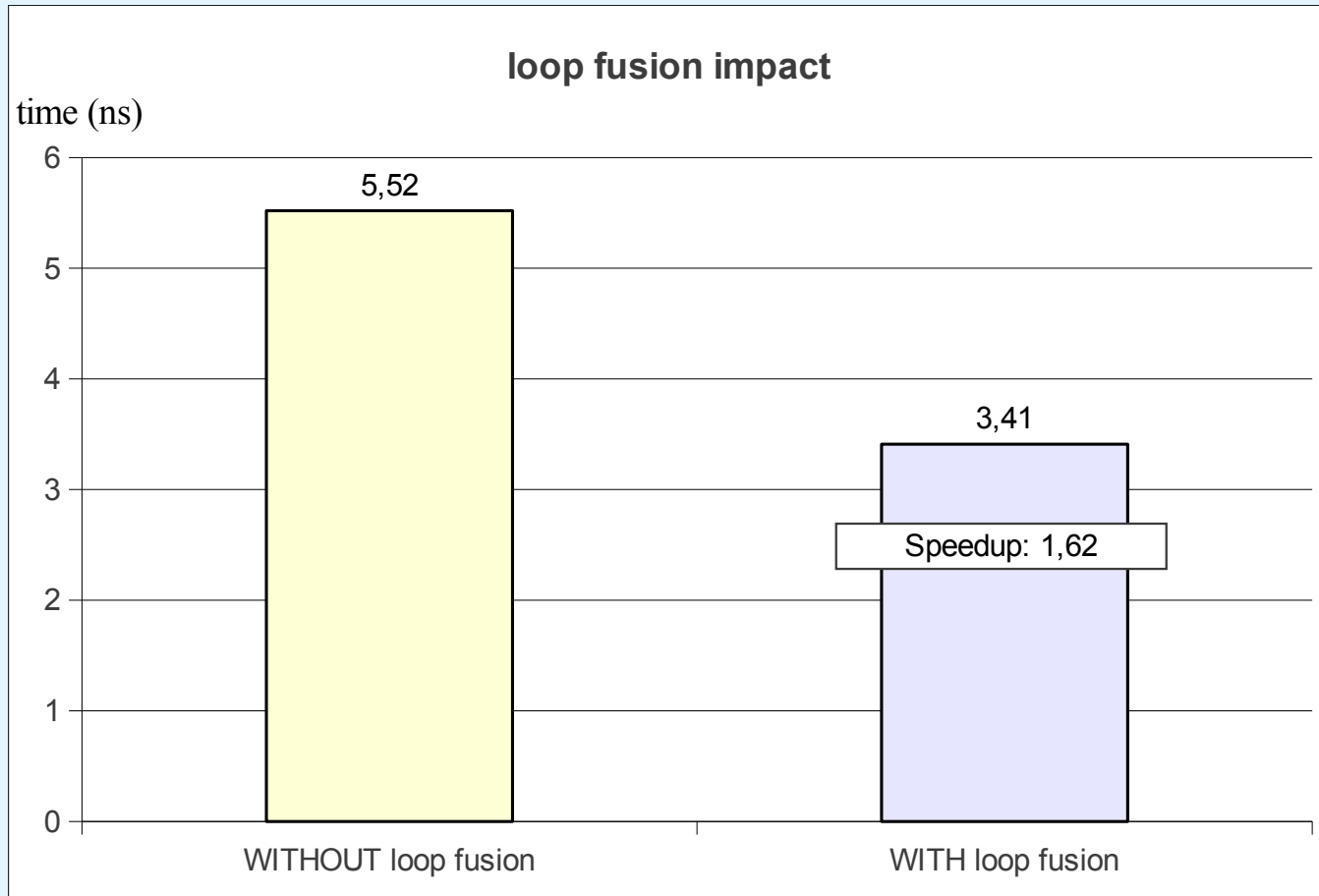
- ***Loop fusion***: combines loops that use the same data elements, possibly improving cache performance.

```
// WITHOUT loop fusion
for (i=0; i<3000; i++)
    C[i] = A[i] + B[i];
for (i=0; i<3000; i++)
    D[i] = E[i] + C[i];
```

```
// WITH loop fusion
for (i=0; i<3000; i++){
    C[i] = A[i] + B[i];
    D[i] = E[i] + C[i];
}
```

7.4 CPU Performance Optimization

- Code Optimization Techniques

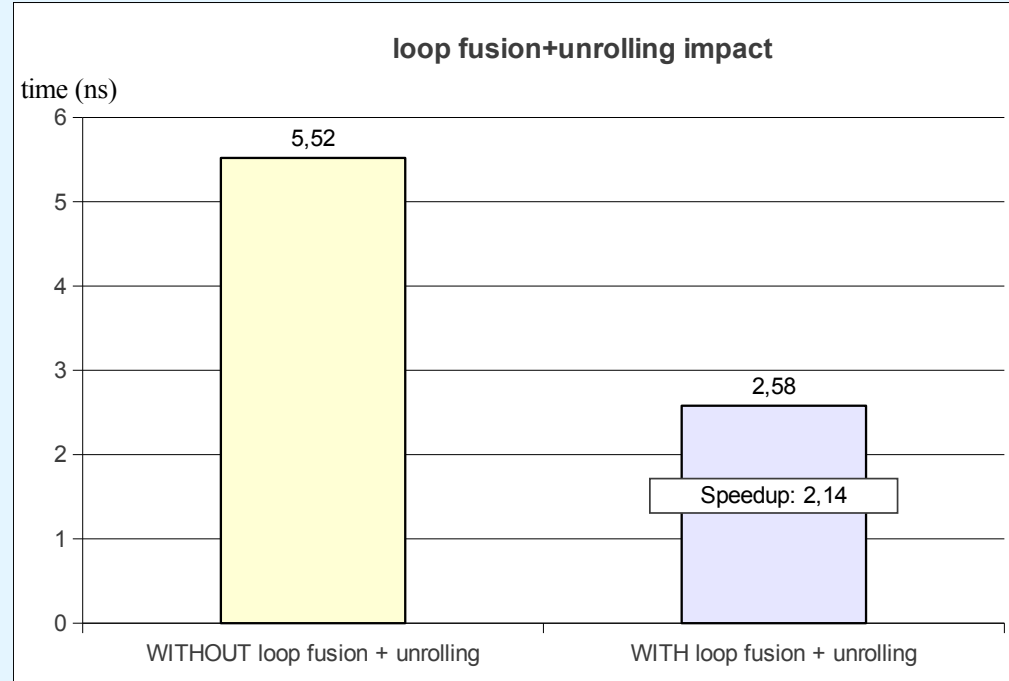


7.4 CPU Performance Optimization

- Code Optimization Techniques

- *Loop fusion + unrolling*

```
// COM loop fusion+unrolling
for (i=0; i<3000; i+=3){
    C[i] = A[i] + B[i];
    C[i+1] = A[i+1] + B[i+1];
    C[i+2] = A[i+2] + B[i+2];
    D[i] = E[i] + C[i];
    D[i+1] = E[i+1] + C[i+1];
    D[i+2] = E[i+2] + C[i+2];
}
```



gcc options -O0 -lm, Core i7-8865U 4.8GHz 14 nm (2018)

7.4 CPU Performance Optimization

- Code Optimization Techniques

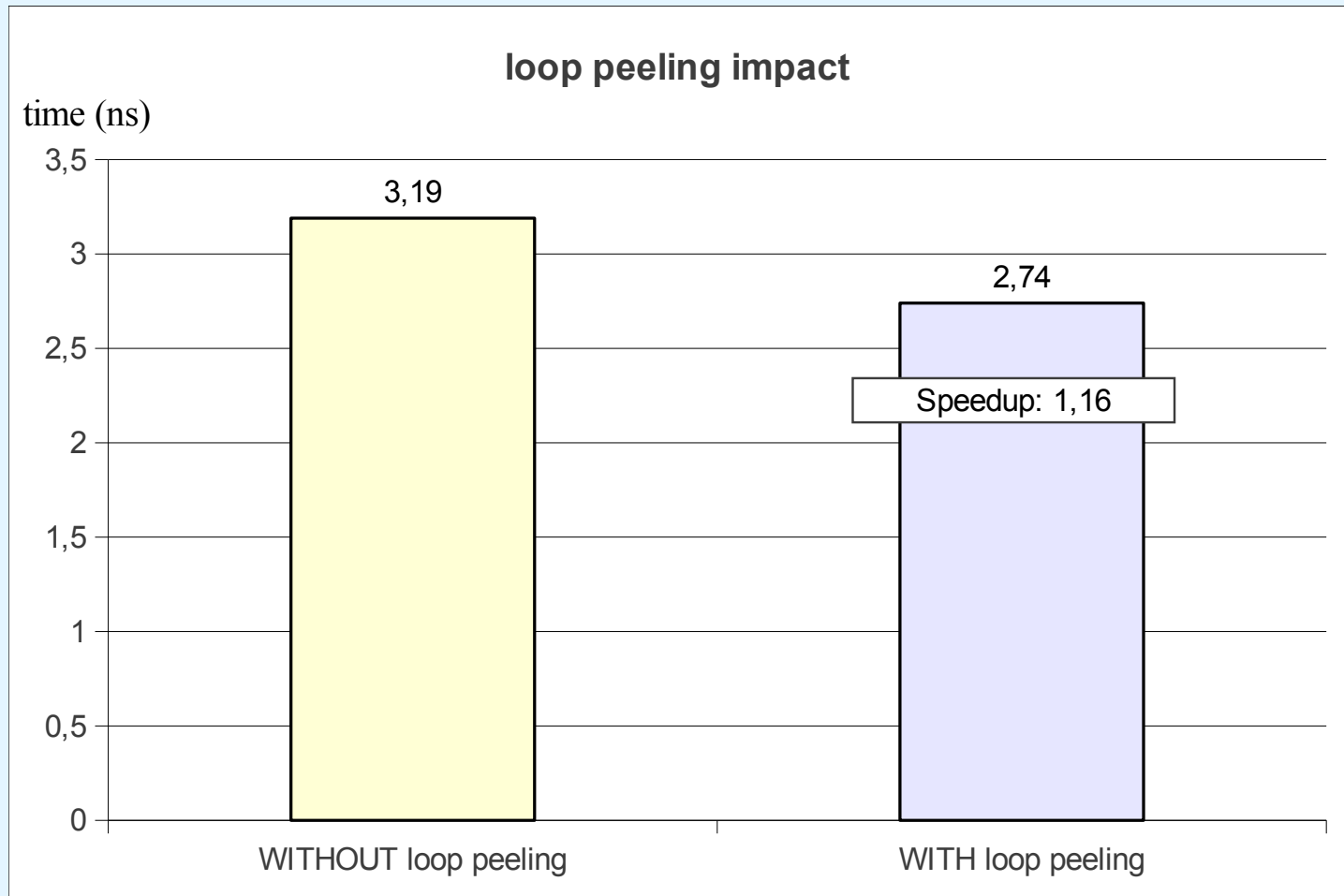
- **Loop fission**: splits large loops into smaller ones to reduce data dependencies and resource conflicts.
- A loop fission technique known as **loop peeling** removes the beginning and ending loop statements.

```
// WITHOUT loop peeling
for (i=0; i<3000; i++) {
    if (i == 0)
        A[i] = 0;
    else if (i == 2999)
        A[i] = 2999;
    else
        A[i] += 8;
}
```

```
// WITH loop peeling
A[0] = 0;
for (i=1; i<2999; i++)
    A[i] += 8;
A[2999] = 2999;
```

7.4 CPU Performance Optimization

- Code Optimization Techniques



7.4 CPU Performance Optimization

- Code Optimization Techniques

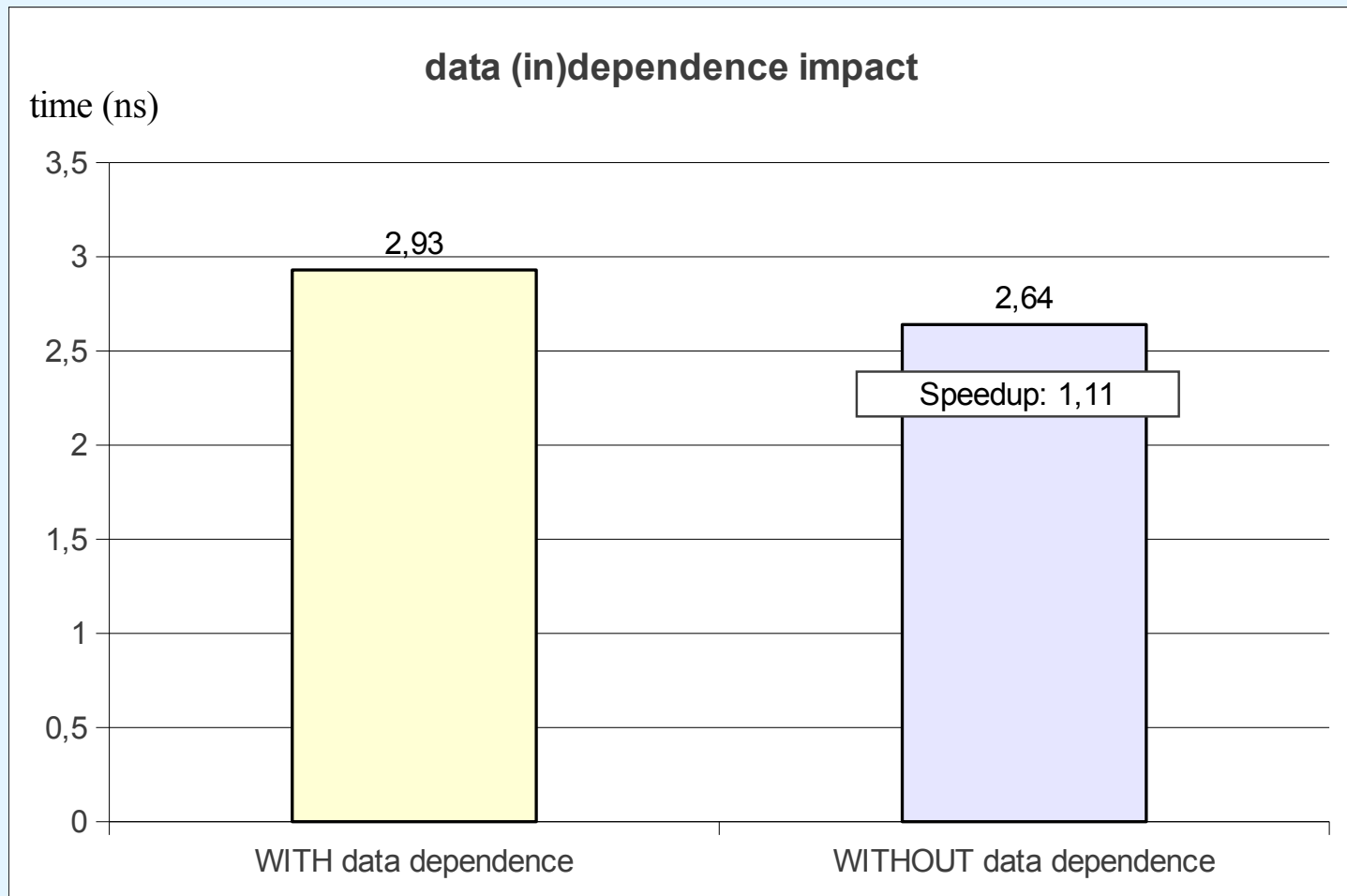
- ***Data (in)dependence***: minimize data dependencies (by changing the order of some independent code)

```
// WITH dependencies
for (i=0; i<3000; i++){
    a1 += a2;
    a3 = a1;
    a4 = a5;
}
```

```
// WITHOUT dependencies
for (i=0; i<3000; i++){
    a1 += a2;
    a4 = a5;
    a3 = a1;
}
```

7.4 CPU Performance Optimization

- Code Optimization Techniques



7.4 CPU Performance Optimization

- Code Optimization Techniques

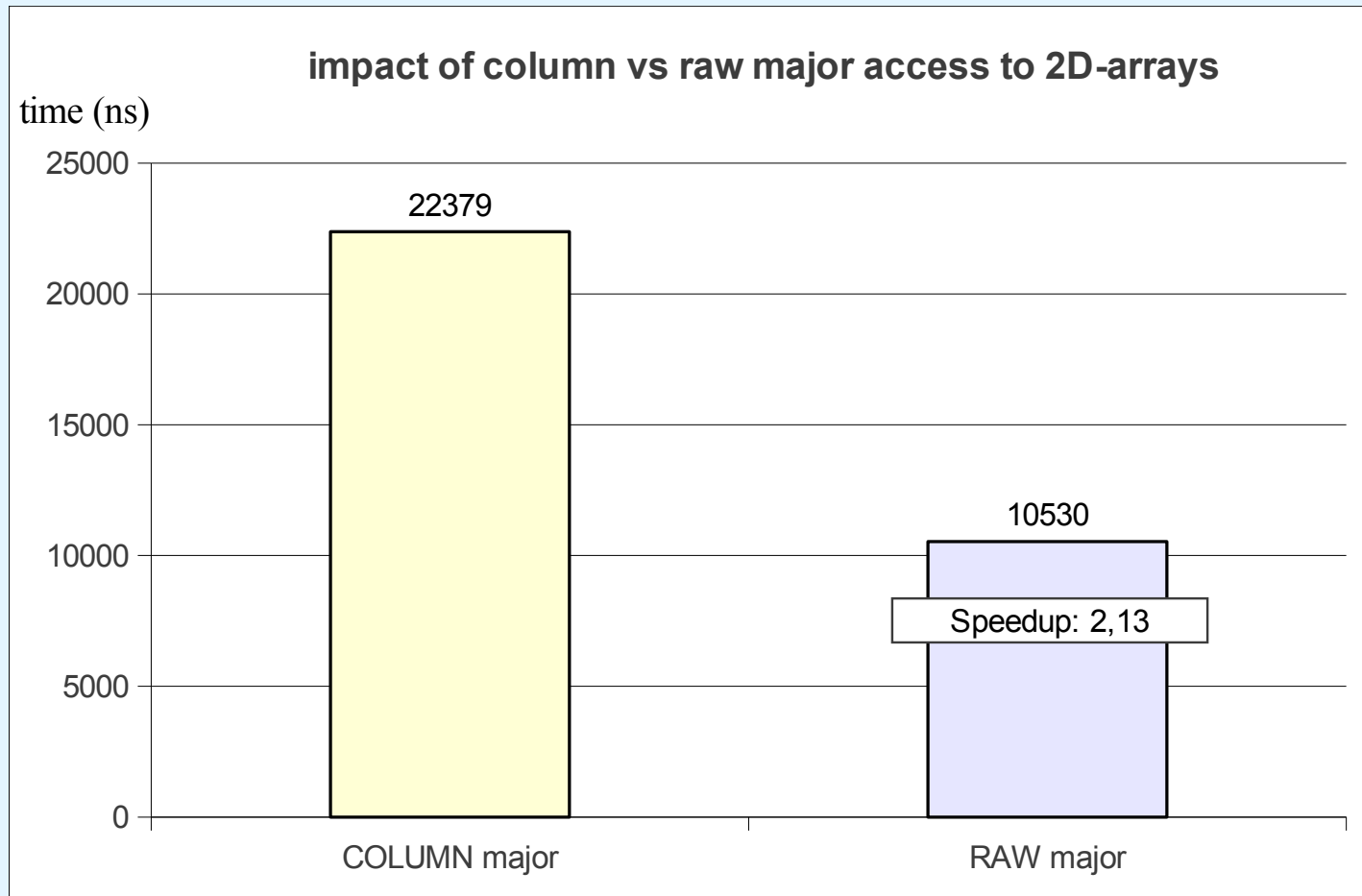
- ***Loop interchange***: involves reorganizing loops for a more efficient access to memory; example: COLUMN-major memory accesses vs ROW-major accesses

```
// COLUMN-major access  
for (i=0; i<3000; i++)  
    for (j=0; j<3000; j++)  
        x[j][i]=0;
```

```
// ROW-major access  
for (i=0; i<3000; i++)  
    for (j=0; j<3000; j++)  
        x[i][j]=0;
```

7.4 CPU Performance Optimization

- Code Optimization Techniques



7.4 CPU Performance Optimization

- Code Optimization Techniques

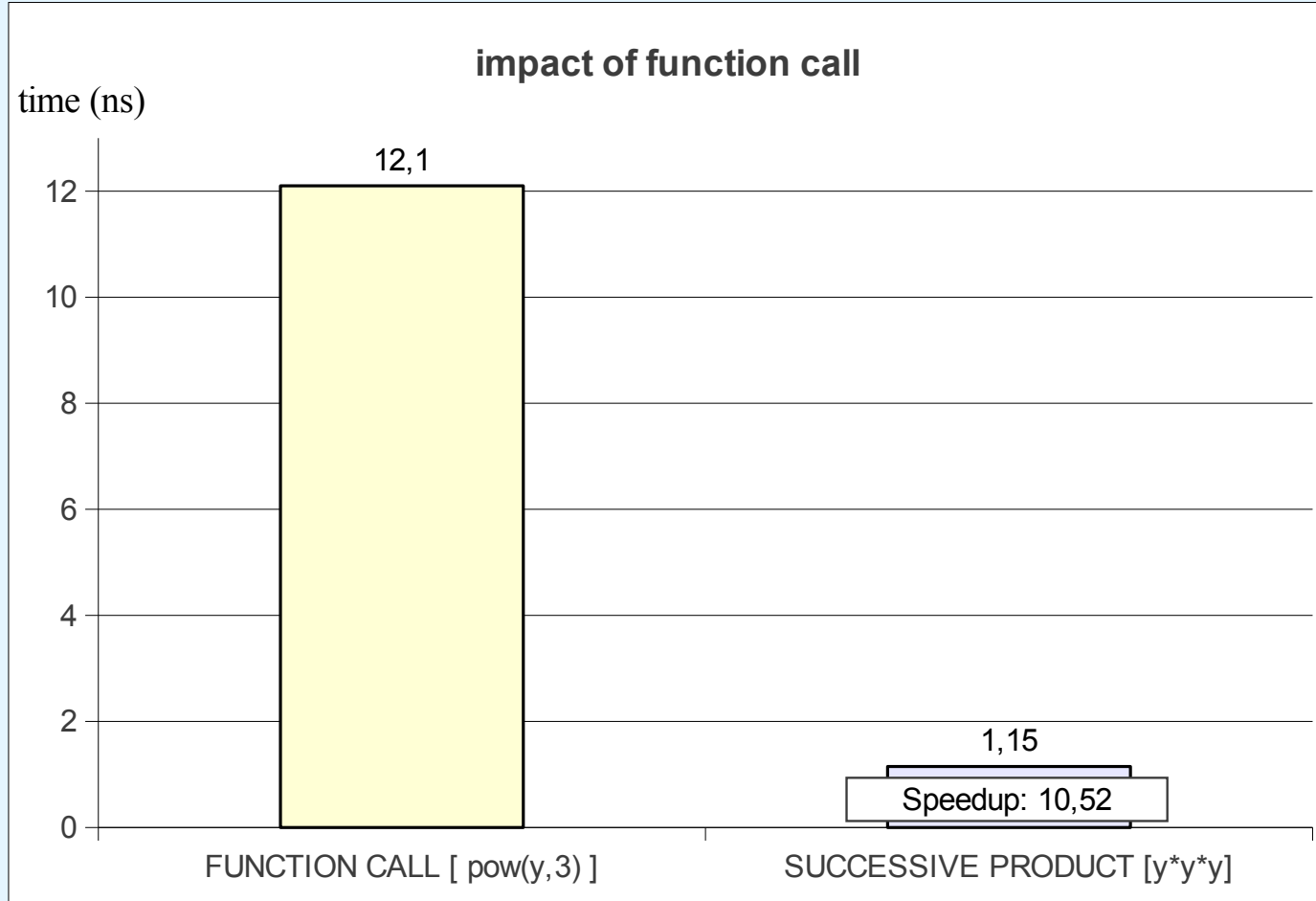
- ***Function call avoidance***: avoids calling routines for tasks that may be implemented locally, thus avoiding the overload of function call and function return

```
// pow function to compute i^3
for (i=0; i<3000; i++)
    y += pow(i, 3);
```

```
// successive product to compute i^3
for (i=0; i<3000; i++)
    y += i * i * i;
```

7.4 CPU Performance Optimization

- Code Optimization Techniques



checkpoint:
activity
TOC1

7.4 CPU Performance Optimization

- Compiler Support

- **compilers** support **options** that have influence over space and time, during compilation and execution
- example: `gcc -Ooptimization_level -funroll-loops prog.c`
 - **-O0** (default): no optimizations;
the best option when debugging is intended
 - **-O1**: optimize but keep compile time low
 - **-O2**: all optimizations that maximize execution speed without making the executable larger; ideal for distributions
 - **-O3**: + (?) performance but + code (*function inlining*, etc.)
 - **-funroll-loops**: + (?) performance, with + code

7.4 CPU Performance Optimization

- Compiler Support

```
#include <stdio.h>

double powern (double d, unsigned n)
{
    double x = 1.0;
    unsigned j;

    for (j = 1; j <= n; j++)
        x *= d;

    return x;
}

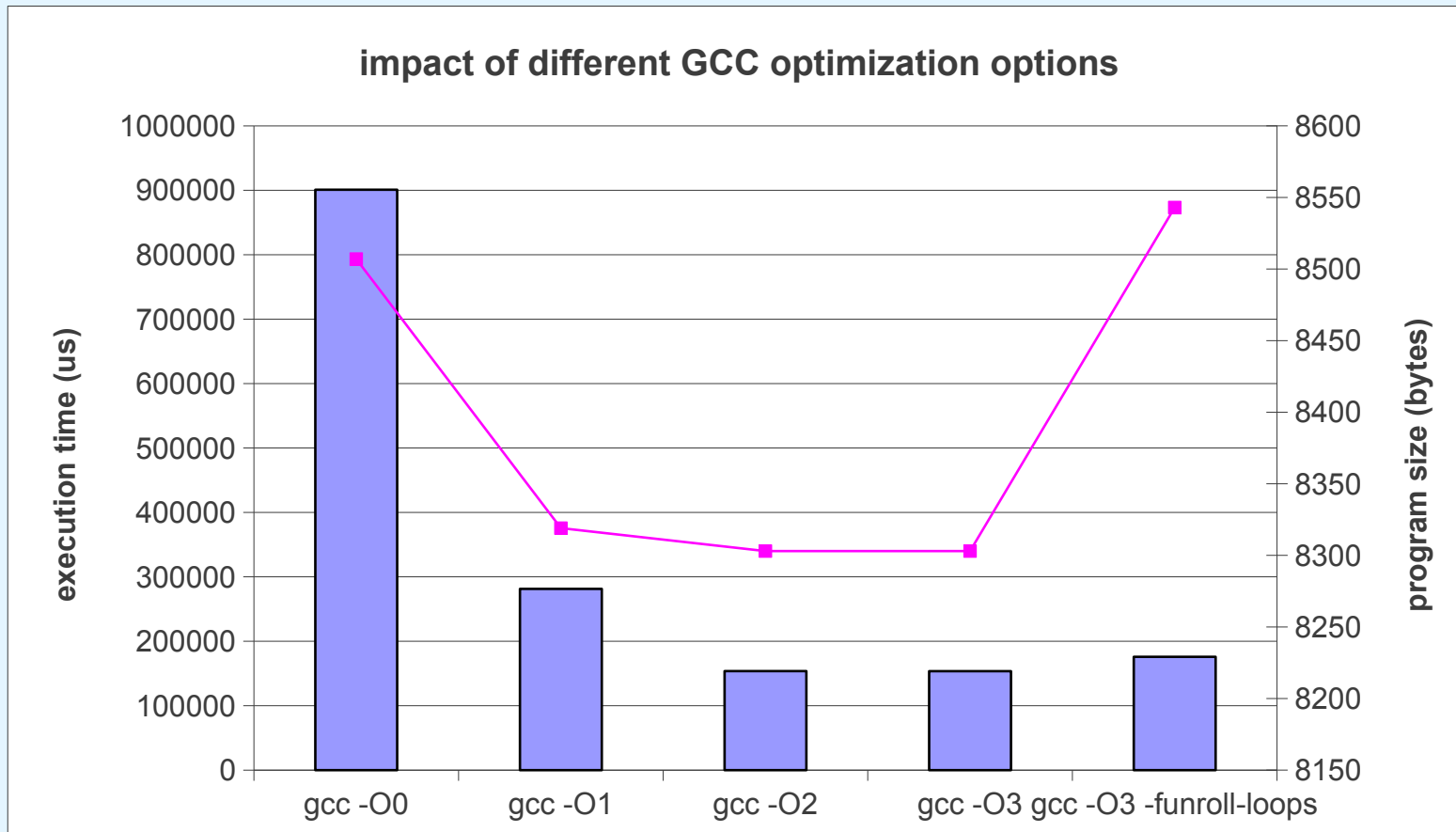
int main (void)
{
    double sum = 0.0;
    unsigned i;

    for (i = 1; i <= 100000000; i++)
    {
        sum += powern (i, i % 5);
    }

    printf ("sum = %g\n", sum);
    return 0;
}
```

7.4 CPU Performance Optimization

- Compiler Support



Core i7-2600 3.8GHz 32nm (2011)

checkpoint:
activity SC1

7.5 Amdahl's Law



- The overall performance of a system is a result of the interaction of all of its components.
- System performance is most effectively improved when the performance of the most heavily used components is improved.
- This idea is quantified by **Amdahl's Law**:

$$S = \frac{1}{(1-f) + \frac{f}{k}}$$

S : the overall speedup

f : the fraction of work performed by the component to be replaced

k : the specific speedup of the replaced component

7.5 Amdahl's Law



- **Amdahl's Law** allows to estimate the change in the overall system performance brought by intervening (upgrade/downgrade) on specific system components
 - It also allows to perform **cost-benefit analysis**
- **example:**
 - On a large system, suppose we can upgrade a CPU to make it 50% faster for \$10,000 or upgrade its disk drives for \$7,000 to make them 150% faster.
 - Programs spend 70% of their time running in the CPU and 30% of their time waiting for disk service.
 - Which upgrade would offer the greater overall performance benefit ? And which would pay off best ?

7.5 Amdahl's Law



- The processor upgrade option offers a speedup $S=1.30=130\%$:

$$\begin{array}{l} f = 0.70, \\ k = 1.5 \end{array} \quad S = \frac{1}{(1 - 0.7) + 0.7/1.5}$$

- And the disk drive upgrade option gives a speedup $S=1.22=122\%$:

$$\begin{array}{l} f = 0.30, \\ k = 2.5 \end{array} \quad S = \frac{1}{(1 - 0.3) + 0.3/2.5}$$

- **Cost-Benefit Analysis:** each 1% of improvement by upgrading the processor has a cost of $\$10000 / 30\% = \333 , and each 1% of improvement by upgrading the disk costs $\$7000 / 22\% = \318 .

Should price/performance be your only concern?

What about aging (obsolescence) of components ?

7.5 Amdahl's Law

- Amdahl's Law also allows to estimate the **impact of simultaneous changes in several components**.
- For the previous example, upgrading the CPU and the Disks at the same time yields a global speedup

$$S = 1 / (f_{\text{CPU}} / k_{\text{CPU}} + f_{\text{Disks}} / k_{\text{Disks}})$$
$$= 1 / (0.7 / 1.5 + 0.3 / 2.5) = 1.70 = 170\%$$

- Generically, with **n** components, if **m** $\leq$ **n** are modified simultaneously, the global speedup is

$$S = 1 / [(1 - (f_1 + \dots + f_m)) + f_1/k_1 + \dots + f_m/k_m]$$
$$= 1 / [(f_{m+1} + \dots + f_n) + f_1/k_1 + \dots + f_m/k_m]$$

with $f_1 + f_2 + \dots + f_m + f_{m+1} + \dots + f_n = 1$

7.5 Amdahl's Law

checkpoint:
exercises LA1 to LA4

- In short, using Amdahl's Law we know that we need to make the common case fast.
- So, if our system is *CPU bound*, we want to make the CPU faster.
- A *memory bound* system calls for improvements in memory management.
- The performance of an *I/O bound* system will improve with an upgrade to the I/O system.

Of course, fixing a performance problem in one part of the system can expose a weakness in another part of the system!

Conclusion



- Computer performance assessment relies upon measures of central tendency that include the arithmetic, the weighted arithmetic, the geometric, and the harmonic mean.
- Benchmark suites have been designed to provide objective performance assessment. The most well respected of these are the SPEC and TPC benchmarks.
- CPU performance depends upon many factors.
- Code optimization methods include loop manipulation and good algorithm design.
- Amdahl's Law quantifies the impact of HW replacements.

References



- [0] ECOA book (6th Ed.), Chapters 7 and 10
- [1] “On Average, You’re Using the Wrong Average: Geometric & Harmonic Means in Data Analysis”,
<https://towardsdatascience.com/on-average-youre-using-the-wrong-average-geometric-harmonic-means-in-data-analysis-2a703e21ea0>
- [2] “SPEC Benchmarks”, <https://www.spec.org/benchmarks.html>
- [3] “TPC-Homepage”, <https://www.tpc.org/>
- [4] “GCC Options That Control Optimization”,
<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- [5] “8 Ways to Measure Execution Time in C/C++”,
<https://levelup.gitconnected.com/8-ways-to-measure-execution-time-in-c-c-48634458d0f9>